**23CSE202 DATABASE MANAGEMENT SYSTEMS**
**S3 B.Tech CSE**
**Department of Computer Science and Engineering**
**Amrita School of Computing, Amritapuri Campus**

# PROJECT REPORT

Group No : C2
Team Members :

| Roll No | Student Name |
|---|---|
| AM.SC.U4CSE25205 | Arjun S P |
| AM.SC.U4CSE25217 | Gayathri Rajeev |
| AM.SC.U4CSE25235 | N Samiksha |
| AM.SC.U4CSE25247 | Sarvesh Sudheer |

Project Title : **Agriculture Supporter: A Location-Based Crop and Livestock Recommendation System**

## CONTENTS


| Sl. No | Chapter | Page No |
|---|---|---|
| 1 | Abstract | 3 |
| 2 | Introduction | 4 |
| 3 | Requirements (Functional & Non-functional) | 6 |
| 4 | UI Design | 7 |
| 5 | Database Design | 9 |
| 6 | Database Selection & Justification | 14 |
| 7 | Implementation | 15 |
| 8 | Testing & Validation | 21 |
| 9 | Results & Screenshots | 23 |
| 10 | Challenges faced & solutions | 29 |
| 11 | Conclusion & future enhancements | 30 |
| | References | 31 |

# 1. Abstract

Farmers in India often have to decide what to grow and which animals to rear without a single, reliable and location-specific source of guidance. Cultivation practices, livestock management advice and local agro-climatic conditions are scattered across leaflets, websites and word of mouth. **Agriculture Supporter** is a database-driven web application built to address this problem. Its objectives are to store agricultural knowledge in a normalised relational database, to give registered users structured guides for crops and livestock, and to recommend suitable crops and animals for a chosen location using its rainfall, soil, climate and water availability. The proposed solution has three tiers: a responsive Bootstrap front end hosted on GitHub Pages, a Node.js and Express REST API hosted on Render, and a PostgreSQL database hosted on Neon. The database contains ten tables in third normal form, including two bridge tables, *location_crop_suitability* and *location_livestock_suitability*, that rate each crop or animal for each location together with the reason. Major database features include primary and foreign keys, UNIQUE, NOT NULL and CHECK constraints, cascading deletes, parameterised multi-table join queries, transactional data loading and bcrypt-hashed credentials. The implemented system holds 14 locations in 6 states, 23 crops in 9 categories and 16 livestock types in 8 categories, with 33 stored recommendations. A user can log in, browse guides, pick a state, district and area, and instantly receive ranked crop and livestock suggestions, and can ask the AgriBot assistant questions that are answered from the database and saved in a chat history.

# 2. Introduction

## 2.1 Problem domain and background

Agriculture remains the main source of livelihood for a large share of India's population, and the decisions a farmer makes before each season largely decide the income that follows. Those decisions depend on many connected factors: the rainfall of the area, the type of soil, the climate, the availability of water, the season, and the market. Livestock rearing, such as dairy, goat, poultry and fish farming, is an equally important source of income, particularly for small and marginal farmers, and it also depends on local climate and water conditions.

India spans many agro-ecological regions. A crop that does well in the waterlogged Kole lands of Thrissur in Kerala is unsuited to the cool highlands of Idukki or the arid plains of Rajasthan. Choosing correctly therefore needs knowledge that is specific to a place, not general advice.

## 2.2 Existing scenario and its limitations

At present a farmer usually gathers information from extension officers, printed booklets, general agricultural websites, mobile applications and neighbours. These sources have several limitations:

- **Not location-specific.** Most sources describe a crop in general terms and do not say whether it suits a particular district or village.
- **Scattered and unstructured.** Crop guides, livestock guides and climate details are held in different places and formats, so they cannot be searched or compared easily.
- **No reasoning shown.** Even when a suggestion is given, the reason (for example high rainfall and waterlogged soil favour rice) is not stated, so the user cannot judge it.
- **No personal history.** Questions asked and answers received are not stored, so the user has to ask again each time.
- **Weak data integrity.** Information kept in spreadsheets or static pages is easily duplicated, contradicted or left out of date, because there are no constraints that keep it consistent.

## 2.3 Proposed system

The proposed database application, Agriculture Supporter, keeps all agricultural knowledge in one relational database and exposes it through a simple web interface. Crops and livestock are organised into categories, each location carries an agro-climatic profile, and bridge tables record how suitable each crop or animal is for each location and why. The application therefore answers the question that matters most to the farmer: *what can I grow or rear here?*

The main objectives of the project are:

- to design a normalised relational schema for crops, livestock, locations, users and conversations;
- to provide secure registration and login with hashed passwords and token-based sessions;

- to provide browsable guides for crop production and animal husbandry;
- to recommend crops and livestock for a chosen location, ranked by suitability with reasons;
- to provide a chat assistant whose answers come from the database and whose history is stored; and
- to deploy the complete system online so that it can be used from any browser.

**Scope.** The reference data (rainfall, soil, suitability ratings) are demonstration values compiled for 14 representative locations in six states. The design allows more locations, crops and animals to be added without changing the schema.

# 3. Requirements (Functional and Non-functional)

## 3.1 Functional requirements

Table 3.1: Functional requirements

| ID | Requirement | Database operation |
|---|---|---|
| FR1 | A new user can register with name, email and password. Duplicate emails are rejected. | INSERT, UNIQUE check |
| FR2 | A registered user can log in and receives a token valid for seven days. | SELECT, password verification |
| FR3 | Pages other than the landing and login pages are available only to logged-in users. | Token check |
| FR4 | The user can view all crop categories, the crops in a category, and the full guide for a crop. | SELECT, JOIN |
| FR5 | The user can view all livestock categories, the animals in a category, and the full guide for an animal. | SELECT, JOIN |
| FR6 | The user can choose a state, then a district, then an area (locality) using linked dropdowns. | SELECT DISTINCT, WHERE |
| FR7 | For the chosen location the system shows the agro-climatic profile and the crops and livestock rated for it, best first, with reasons. | Multi-table JOIN, ORDER BY |
| FR8 | The user can ask AgriBot about a crop, an animal or a place and gets an answer taken from the database. | SELECT, pattern matching |
| FR9 | Every question and answer is stored against the user and can be retrieved later. | INSERT, SELECT |
| FR10 | Reference data (crops, animals, locations, suitability) can be inserted, updated and deleted by the administrator using SQL, and is loaded in bulk by a seed script. | INSERT, UPDATE, DELETE, transaction |
| FR11 | Recommendations can optionally be linked to a crop, an animal and a user (personalisation). | Nullable foreign keys |
| FR12 | The system shows a clear message when a location has no stored recommendations. | Empty-result handling |

## 3.2 Non-functional requirements

Table 3.2: Non-functional requirements

| Quality | Requirement and how it is met |
|---|---|
| Security | Passwords are hashed with bcrypt (10 rounds) and never stored in plain text. Sessions use signed JWTs. All SQL uses parameterised queries ($1, $2) to prevent SQL injection. Secrets are kept in environment variables, not in the code. Traffic uses HTTPS and the database connection uses TLS. Cross-origin access is limited to an allow-list. |
| Integrity | Primary keys, foreign keys, UNIQUE, NOT NULL and CHECK constraints keep data consistent. Bulk loading runs inside a transaction that is rolled back on any error. |
| Performance | Typical requests are simple indexed lookups and joins on small tables and respond within a second. A pooled database connection is reused across requests. On the free hosting plan the first request after 15 minutes of inactivity can take up to a minute while the server wakes up. |
| Reliability | The database is a managed cloud service. The API restarts automatically after deployment or failure. Errors return a JSON message and a suitable HTTP status code instead of crashing. |
| Usability | Responsive Bootstrap layout that works on phones and desktops, linked dropdowns, plain-language messages, and a floating assistant available on every page. |
| Scalability | The API is stateless, so it can be run as several instances. New locations, crops and animals are added as rows. Hosting can be upgraded without changing the code. |
| Maintainability | The front end reaches the data only through one data-access file (data.js). The schema and seed script are plain files that can be re-run on any PostgreSQL server. |

# 4. UI Design

The interface is a set of responsive web pages built with HTML, CSS, Bootstrap 5 and JavaScript. A consistent green and wheat colour scheme is used throughout. The navigation bar on every logged-in page shows the links Dashboard, Animal Husbandry, Crop Production and Search by Location, the user's first name and a Log out button. A floating AgriBot button is available at the bottom right of every page.

## 4.1 Navigation flow

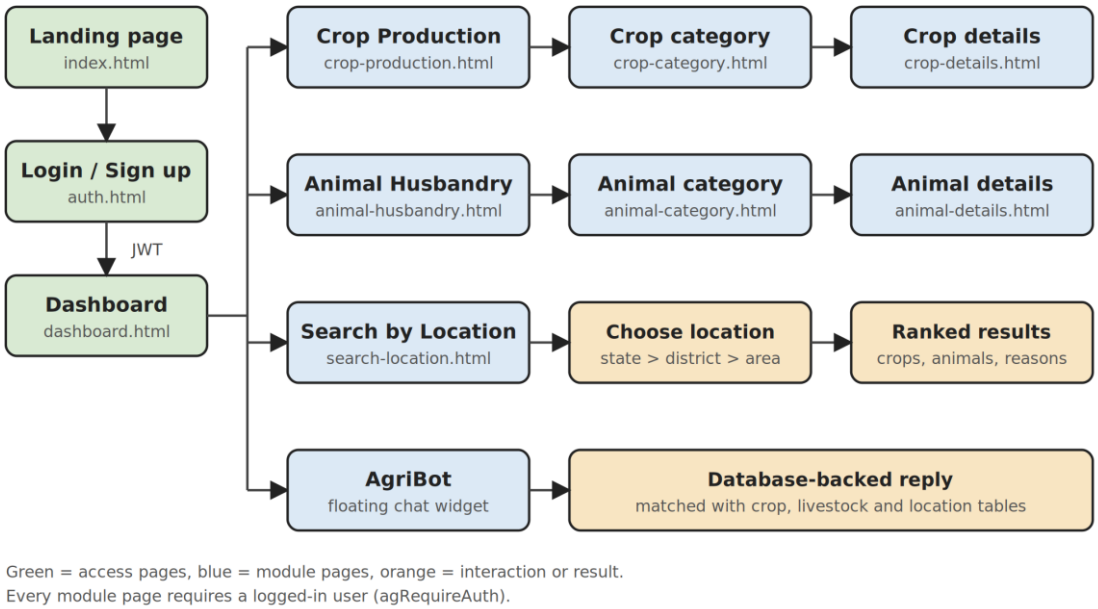



Figure 4.1: Navigation flow of the application

A visitor starts on the landing page, moves to the login or signup page, and on success receives a token and is taken to the dashboard. From the dashboard the user can open crop production, animal husbandry or location search, and can open the chat assistant on any page.

## 4.2 Screen design

Table 4.1: Main screens, their purpose and the data they use

| Screen (file) | Design and interaction | Data used |
|---|---|---|
| Landing page (index.html) | Hero section with the project message, a Get Started button, key figures and a feature overview. | None (static) |
| Login / Sign up (auth.html) | Two tabs. Login: email and password. Sign up: full name, email, password, confirm password and primary interest. Errors appear in a coloured message box; a spinner shows while the request is running. | users |
| Dashboard (dashboard.html) | Greets the user by first name with today's date and shows cards that lead to the three modules. | Session (name) |
| Crop Production (crop-production.html) | A grid of nine category cards, each with an icon, colour and short description. | crop_category |
| Crop category (crop-category.html) | Lists the crops of the selected category with their season and water need. | crop, crop_category |

| Screen (file) | Design and interaction | Data used |
|---|---|---|
| Crop details (crop-details.html) | Full guide with growth conditions, climate, soil, fertiliser, irrigation, pest management, harvesting and season. | crop |
| Animal Husbandry, category and details (animal-*.html) | Same three-level layout for livestock: care, feeding, diseases, housing, breeding and management. | livestock_category, livestock |
| Search by Location (search-location.html) | Three linked dropdowns: State, District, Area. The next dropdown is filled only after the previous one is chosen. A Search button shows the result panel. | location |
| Location result (same page) | Header with the place name and climate badge, a line with agro zone, soil, rainfall and water availability, then cards for crops and for livestock. Each card has a suitability badge (Highly Suitable, Suitable, Moderately Suitable) and the reason. | location, suitability tables, crop, livestock |
| AgriBot (chatbot.js) | Floating button opens a chat panel with quick-question chips. Messages appear as bubbles and the reply is fetched from the server. | crop, livestock, location, chat_history |

### 4.3 How users interact with the system

- **Data entry:** the signup form and the chat box send data that is stored in the database.
- **Search:** the linked dropdowns narrow the choice from state to district to locality, so the user cannot enter a location that does not exist in the database.
- **Report generation:** the location result panel acts as an on-screen recommendation report, ordered from most to least suitable, with the reason for each rating.
- **Update and delete:** reference data is maintained by the administrator through SQL, as described in Chapter 7. Users cannot change reference data.

**Screenshots** of each screen are given in Chapter 9.

# 5. Database Design

## 5.1 ER diagram

Figure 5.1 shows the final ER diagram. Green boxes are main tables and tan boxes are dependent or bridge tables. Solid lines are mandatory links and dashed lines are optional links, which are implemented as foreign keys that may be NULL. A full-size copy is supplied with the report as ER_Diagram.png.

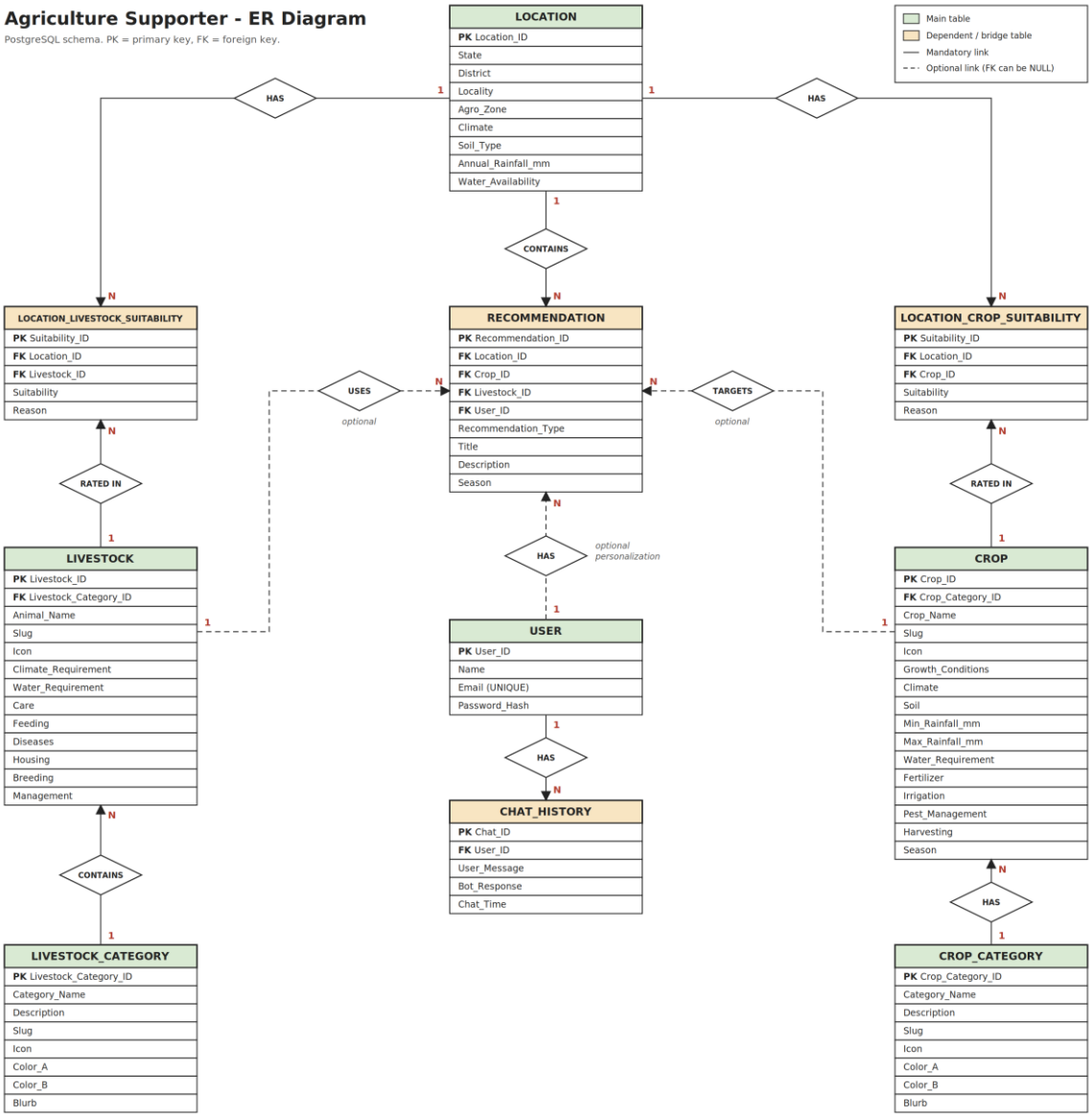



Figure 5.1: ER diagram of Agriculture Supporter

## 5.2 Entities and attributes

Table 5.1: Entities and attributes (PK = primary key, FK = foreign key)

| Entity (table) | Attributes | Purpose |
|---|---|---|
| USER (users) | **user_id** (PK), name, email (UNIQUE), password_hash | Registered user of the system. |
| CHAT_HISTORY (chat_history) | **chat_id** (PK), user_id (FK), user_message, bot_response, chat_time | One question and answer exchanged with AgriBot. |
| LOCATION (location) | **location_id** (PK), state, district, locality, agro_zone, climate, soil_type, annual_rainfall_mm, water_availability | An area with its agro-climatic profile. |
| CROP_CATEGORY (crop_category) | **crop_category_id** (PK), category_name, description, slug, icon, color_a, color_b, blurb | Group of crops such as Cereals or Spices. |
| CROP (crop) | **crop_id** (PK), crop_category_id (FK), crop_name, slug, icon, growth_conditions, climate, soil, min_rainfall_mm, max_rainfall_mm, water_requirement, fertilizer, irrigation, pest_management, harvesting, season | Cultivation guide and requirements of a crop. |
| LIVESTOCK_CATEGORY (livestock_category) | **livestock_category_id** (PK), category_name, description, slug, icon, color_a, color_b, blurb | Group of animals such as Dairy or Poultry. |
| LIVESTOCK (livestock) | **livestock_id** (PK), livestock_category_id (FK), animal_name, slug, icon, climate_requirement, water_requirement, care, feeding, diseases, housing, breeding, management | Management guide and requirements of an animal. |
| LOCATION_CROP_SUITABILITY | **suitability_id** (PK), location_id (FK), crop_id (FK), suitability, reason | Rating of a crop for a location. |
| LOCATION_LIVESTOCK_SUITABILITY | **suitability_id** (PK), location_id (FK), livestock_id (FK), suitability, reason | Rating of an animal for a location. |
| RECOMMENDATION (recommendation) | **recommendation_id** (PK), location_id (FK), crop_id (FK, nullable), livestock_id (FK, nullable), user_id (FK, nullable), recommendation_type, title, description, season | Generated recommendation text for a location, optionally tied to a crop, an animal and a user. |

## 5.3 Relationships and cardinalities

Table 5.2: Relationships

| Relationship | Entities | Cardinality | Participation | Implemented as |
|---|---|---|---|---|
| HAS | USER, CHAT_HISTORY | 1 : N | Total on CHAT_HISTORY | chat_history.user_id NOT NULL, ON DELETE CASCADE |
| HAS (personalisation) | USER, RECOMMENDATION | 1 : N | Optional | recommendation.user_id, nullable FK |
| CONTAINS | LOCATION, RECOMMENDATION | 1 : N | Total on RECOMMENDATION | recommendation.location_id NOT NULL |
| TARGETS | CROP, RECOMMENDATION | 1 : N | Optional | recommendation.crop_id, nullable FK |
| USES | LIVESTOCK, RECOMMENDATION | 1 : N | Optional | recommendation.livestock_id, nullable FK |
| HAS | CROP_CATEGORY, CROP | 1 : N | Total on CROP | crop.crop_category_id NOT NULL |

| Relationship | Entities | Cardinality | Participation | Implemented as |
|---|---|---|---|---|
| CONTAINS | LIVESTOCK_CATEGORY, LIVESTOCK | 1 : N | Total on LIVESTOCK | livestock.livestock_category_id NOT NULL |
| HAS and RATED IN | LOCATION, CROP via LOCATION_CROP_SUITABILITY | M : N | Optional | Bridge table with two FKs and UNIQUE(location_id, crop_id) |
| HAS and RATED IN | LOCATION, LIVESTOCK via LOCATION_LIVESTOCK_SUITABILITY | M : N | Optional | Bridge table with two FKs and UNIQUE(location_id, livestock_id) |

## 5.4 Conversion of the ER model into relational tables

The following rules were applied to convert the ER diagram into relational tables:

- **Strong entities** (USER, LOCATION, CROP_CATEGORY, CROP, LIVESTOCK_CATEGORY, LIVESTOCK) become tables. Each gets an integer surrogate primary key generated by a SERIAL sequence.
- **One-to-many relationships** are implemented by placing the primary key of the "one" side as a foreign key in the table of the "many" side (for example crop.crop_category_id).
- **Optional participation** is implemented by allowing the foreign key to be NULL. This is why recommendation.crop_id, livestock_id and user_id are nullable, while location_id is NOT NULL.
- **Many-to-many relationships.** A location suits many crops and a crop suits many locations, and the relationship carries its own attributes (suitability and reason). It is therefore converted into a bridge table whose foreign keys are location_id and crop_id. The same is done for livestock.
- **Weak or dependent entities** such as CHAT_HISTORY depend on USER and use ON DELETE CASCADE, so a user's chats are removed with the user.

## 5.5 Relational schema

```
USERS(user_id PK, name, email UNIQUE, password_hash)
CHAT_HISTORY(chat_id PK, user_id FK -> USERS, user_message, bot_response, chat_time)
LOCATION(location_id PK, state, district, locality, agro_zone, climate, soil_type,
         annual_rainfall_mm, water_availability)
CROP_CATEGORY(crop_category_id PK, category_name, description, slug, icon,
              color_a, color_b, blurb)
CROP(crop_id PK, crop_category_id FK -> CROP_CATEGORY, crop_name, slug, icon,
     growth_conditions, climate, soil, min_rainfall_mm, max_rainfall_mm,
     water_requirement, fertilizer, irrigation, pest_management, harvesting, season)
LIVESTOCK_CATEGORY(livestock_category_id PK, category_name, description, slug, icon,
                   color_a, color_b, blurb)
LIVESTOCK(livestock_id PK, livestock_category_id FK -> LIVESTOCK_CATEGORY, animal_name,
          slug, icon, climate_requirement, water_requirement, care, feeding, diseases,
          housing, breeding, management)
LOCATION_CROP_SUITABILITY(suitability_id PK, location_id FK -> LOCATION,
          crop id FK -> CROP, suitability, reason, UNIQUE(location id, crop id))
LOCATION_LIVESTOCK_SUITABILITY(suitability_id PK, location_id FK -> LOCATION,
          livestock_id FK -> LIVESTOCK, suitability, reason,
          UNIQUE(location_id, livestock_id))
RECOMMENDATION(recommendation_id PK, location_id FK -> LOCATION,
          crop_id FK -> CROP NULL, livestock_id FK -> LIVESTOCK NULL,
          user_id FK -> USERS NULL, recommendation_type CHECK IN ('crop','livestock'),
          title, description, season)
```

The user table is named *users* in PostgreSQL because USER is a reserved word.

## 5.6 Constraints

Table 5.3: Constraints used in the schema

| Constraint | Where used | Purpose |
|---|---|---|
| PRIMARY KEY (SERIAL) | All ten tables | Unique identity of every row. |
| FOREIGN KEY | crop, livestock, chat_history, recommendation, both suitability tables | No row can refer to a parent that does not exist. |
| UNIQUE | users.email; (location_id, crop_id); (location_id, livestock_id) | One account per email; one rating per location and crop or animal. |
| NOT NULL | Names, keys and the mandatory foreign keys | Mandatory data must be present. |
| CHECK | recommendation.recommendation_type IN ('crop', 'livestock') | Only valid recommendation types are stored. |
| ON DELETE CASCADE | chat_history.user_id | Deleting a user also deletes that user's chats. |
| DEFAULT | chat_history.chat_time DEFAULT NOW() | The time of a chat is recorded automatically. |

## 5.7 Functional dependencies and normalisation

**First normal form (1NF).** Every column holds a single value for a row. Descriptive fields such as pest_management hold one piece of descriptive text and are not split into repeating groups. There are no repeating columns such as crop1, crop2. A location's several suitable crops are rows in a bridge table, not a list inside the location row.

**Second normal form (2NF).** Each table has a single-column primary key, so no non-key attribute can depend on part of a key. In the bridge tables, suitability and reason depend on the whole pair (location, crop) or (location, animal).

**Third normal form (3NF) and BCNF.** No non-key attribute depends on another non-key attribute. Table 5.4 lists the functional dependencies (FDs).

Table 5.4: Functional dependencies and normal form of each table

| Table | Functional dependencies | Candidate keys | Normal form |
|---|---|---|---|
| users | user_id → name, email, password_hash<br>email → user_id | user_id, email | BCNF |
| chat_history | chat_id → user_id, user_message, bot_response, chat_time | chat_id | BCNF |
| location | location_id → state, district, locality, agro_zone, climate, soil_type, annual_rainfall_mm, water_availability<br>(state, district, locality) → location_id | location_id, (state, district, locality) | BCNF |
| crop_category | crop_category_id → category_name, description, slug, icon, color_a, color_b, blurb | crop_category_id | BCNF |
| crop | crop_id → all other attributes, including crop_category_id | crop_id | BCNF |
| livestock_category | livestock_category_id → category_name, description, slug, icon, color_a, color_b, blurb | livestock_category_id | BCNF |

| Table | Functional dependencies | Candidate keys | Normal form |
|---|---|---|---|
| livestock | livestock_id → all other attributes, including livestock_category_id | livestock_id | BCNF |
| location_crop_suitability | suitability_id → location_id, crop_id, suitability, reason<br>(location_id, crop_id) → suitability_id, suitability, reason | suitability_id, (location_id, crop_id) | BCNF |
| location_livestock_suitability | suitability_id → location_id, livestock_id, suitability, reason<br>(location_id, livestock_id) → suitability_id, suitability, reason | suitability_id, (location_id, livestock_id) | BCNF |
| recommendation | recommendation_id → location_id, crop_id, livestock_id, user_id, recommendation_type, title, description, season | recommendation_id | 2NF (see note) |

**Note on RECOMMENDATION.** The columns title, description and season are generated from the crop, location and suitability data when a recommendation row is created, so they repeat information held elsewhere. The table is therefore a deliberate, controlled denormalisation, a stored snapshot of generated recommendations, and it is in 2NF rather than 3NF. The normalised source of truth is the pair of suitability tables. If strict 3NF is required, the table can be replaced by a view:

```
CREATE VIEW v crop recommendation AS
SELECT l.location_id, l.locality, c.crop_name, c.season, s.suitability, s.reason
FROM location_crop_suitability s
JOIN location l ON l.location_id = s.location_id
JOIN crop c     ON c.crop_id     = s.crop_id;
```

**Note on LOCATION.** The attribute agro_zone does not determine soil_type or annual_rainfall_mm, because the High Range zone appears at both Idukki and Wayanad with different soil and rainfall. These values depend on the location itself, so there is no transitive dependency.

***Redundancy and anomalies avoided***

- **Update anomaly:** a category name is stored once in crop_category. Renaming Cereals changes one row and not every cereal crop.
- **Insertion anomaly:** a new category can be added before it has any crops, and a new location can be added before any crop is rated for it.
- **Deletion anomaly:** deleting a rating removes only that rating and not the crop or the location. Deleting a user removes only that user's chats.
- **Redundancy:** the long guide text of a crop is stored once and is reused by every location that rates the crop.

## 6. Database Selection and Justification

**PostgreSQL** was selected as the DBMS for Agriculture Supporter. It was used on a local installation (managed with pgAdmin 4) during development and as a managed cloud database on Neon for the deployed system.

The factors that decided the choice were:

- **Integrity features.** The design relies on foreign keys, composite UNIQUE constraints, CHECK constraints and ON DELETE CASCADE, all of which PostgreSQL supports fully.
- **Standards-compliant SQL.** The joins, subqueries, grouping and views used in the project follow standard SQL, and the course topics on SQL and PL/SQL map directly to PostgreSQL and PL/pgSQL.
- **Cost and ease of use.** It is free, easy to install, comes with the pgAdmin administration tool, and has a free managed cloud option, so the whole system could be deployed at no cost.
- **Security.** It supports encrypted (TLS) connections, which Neon requires, and role-based access.
- **Scalability and support.** It handles far larger data volumes than this project needs, has extensive documentation, and offers connection pooling for web back ends.

# 7. Implementation

## 7.1 Architecture and technology stack

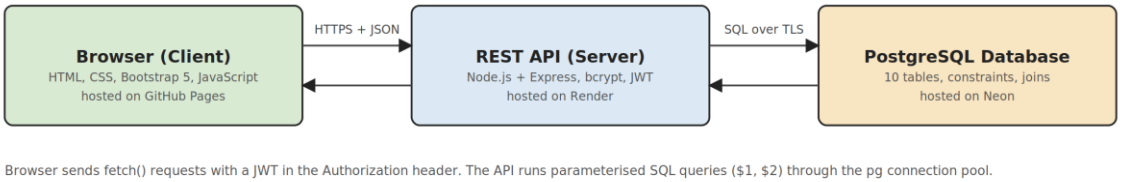



Figure 7.1: Three-tier architecture of the deployed system

Table 7.1: Technology stack

| Layer | Technology | Hosting |
|---|---|---|
| Front end | HTML5, CSS3, Bootstrap 5, JavaScript (fetch API) | GitHub Pages |
| Back end | Node.js, Express, pg (node-postgres), bcrypt, jsonwebtoken, cors, dotenv | Render (free web service) |
| Database | PostgreSQL 18 (pgAdmin 4 for administration) | Local server for development, Neon for production |

## 7.2 Database creation

The database *agriculture_supporter* was created in pgAdmin 4 and the tables were created with the script below. For deployment the same script was run in the Neon SQL Editor.

## 7.3 Table creation with constraints

```
CREATE TABLE users (
  user_id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL
);
CREATE TABLE location (
  location_id SERIAL PRIMARY KEY,
  state VARCHAR(60) NOT NULL, district VARCHAR(60) NOT NULL, locality VARCHAR(80),
  agro_zone VARCHAR(80), climate VARCHAR(80), soil_type VARCHAR(80),
  annual_rainfall_mm INT, water_availability VARCHAR(20)
);
CREATE TABLE crop_category (
  crop_category_id SERIAL PRIMARY KEY,
  category_name VARCHAR(60) NOT NULL, description TEXT,
  slug VARCHAR(60), icon VARCHAR(10), color_a VARCHAR(10), color_b VARCHAR(10), blurb TEXT
);
CREATE TABLE crop (
  crop_id SERIAL PRIMARY KEY,
  crop_category_id INT NOT NULL REFERENCES crop_category(crop_category_id),
  crop_name VARCHAR(80) NOT NULL, slug VARCHAR(80), icon VARCHAR(10),
  growth_conditions TEXT, climate TEXT, soil TEXT,
  min_rainfall_mm INT, max_rainfall_mm INT, water_requirement TEXT,
  fertilizer TEXT, irrigation TEXT, pest_management TEXT, harvesting TEXT, season TEXT
);
CREATE TABLE livestock_category (
  livestock_category_id SERIAL PRIMARY KEY,
  category_name VARCHAR(60) NOT NULL, description TEXT,
  slug VARCHAR(60), icon VARCHAR(10), color_a VARCHAR(10), color_b VARCHAR(10), blurb TEXT
);
CREATE TABLE livestock (
  livestock_id SERIAL PRIMARY KEY,
  livestock_category_id INT NOT NULL
      REFERENCES livestock_category(livestock_category_id),
  animal_name VARCHAR(80) NOT NULL, slug VARCHAR(80), icon VARCHAR(10),
```

```
  climate_requirement TEXT, water_requirement TEXT,
  care TEXT, feeding TEXT, diseases TEXT, housing TEXT, breeding TEXT, management TEXT
);
CREATE TABLE location_crop_suitability (
  suitability_id SERIAL PRIMARY KEY,
  location_id INT NOT NULL REFERENCES location(location_id),
  crop_id INT NOT NULL REFERENCES crop(crop_id),
  suitability VARCHAR(30) NOT NULL, reason TEXT,
  UNIQUE (location_id, crop_id)
);
CREATE TABLE location_livestock_suitability (
  suitability_id SERIAL PRIMARY KEY,
  location_id INT NOT NULL REFERENCES location(location_id),
  livestock_id INT NOT NULL REFERENCES livestock(livestock_id),
  suitability VARCHAR(30) NOT NULL, reason TEXT,
  UNIQUE (location_id, livestock_id)
);
CREATE TABLE recommendation (
  recommendation_id SERIAL PRIMARY KEY,
  location_id INT NOT NULL REFERENCES location(location_id),
  crop_id INT REFERENCES crop(crop_id),
  livestock_id INT REFERENCES livestock(livestock_id),
  user_id INT REFERENCES users(user_id),
  recommendation_type VARCHAR(20) NOT NULL
      CHECK (recommendation_type IN ('crop','livestock')),
  title TEXT NOT NULL, description TEXT, season TEXT
);
CREATE TABLE chat_history (
  chat_id SERIAL PRIMARY KEY,
  user_id INT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
  user_message TEXT NOT NULL, bot_response TEXT,
  chat_time TIMESTAMP DEFAULT NOW()
);
```

### 7.4 Sample data

Locations were inserted directly with SQL. Categories, crops, animals and suitability ratings were loaded by a Node.js seed script that reads the project's data file and inserts everything inside one transaction. If any statement fails, the transaction is rolled back and the database is left unchanged.

```
INSERT INTO location (state, district, locality, agro zone, climate, soil type,
                      annual_rainfall_mm, water_availability) VALUES
('Kerala','Thrissur','Kole Lands','Kole Lands','Tropical humid monsoon',
 'Alluvial / waterlogged',2902,'High'),
('Kerala','Idukki','High Ranges','High Range','Cool humid',
 'Laterite / forest soil',3000,'High'),
('Punjab','Ludhiana','Ludhiana','Trans-Gangetic Plains','Subtropical',
 'Alluvial',700,'High');
-- seed.js (simplified)
BEGIN;
TRUNCATE recommendation, location_crop_suitability, location_livestock_suitability,
         crop, crop_category, livestock, livestock_category RESTART IDENTITY CASCADE;
-- INSERT categories, crops, animals, then suitability rows
COMMIT;      -- or ROLLBACK on any error
```

The recommendation table was then filled from the suitability data with an INSERT ... SELECT query:

```
INSERT INTO recommendation (location_id, crop_id, recommendation_type,
                            title, description, season)
SELECT s.location_id, s.crop_id, 'crop',
       c.crop_name || ' in ' || l.locality,
       s.suitability || ': ' || s.reason, c.season
FROM location_crop_suitability s
JOIN crop c     ON c.crop_id = s.crop_id
JOIN location l ON l.location_id = s.location_id;
```

Row counts after loading:

```
SELECT 'location' AS tbl, COUNT(*) FROM location
UNION ALL SELECT 'crop_category', COUNT(*) FROM crop_category
UNION ALL SELECT 'crop', COUNT(*) FROM crop
UNION ALL SELECT 'livestock_category', COUNT(*) FROM livestock_category
UNION ALL SELECT 'livestock', COUNT(*) FROM livestock
```

```
UNION ALL SELECT 'location_crop_suitability', COUNT(*) FROM location_crop_suitability
UNION ALL SELECT 'location_livestock_suitability', COUNT(*) FROM location_livestock_suitability
UNION ALL SELECT 'recommendation', COUNT(*) FROM recommendation;
```

Output 7.1: Row counts

| tbl | count |
|---|---|
| location | 14 |
| crop_category | 9 |
| crop | 23 |
| livestock_category | 8 |
| livestock | 16 |
| location_crop_suitability | 22 |
| location_livestock_suitability | 11 |
| recommendation | 33 |

## 7.5 CRUD operations

**Create.** Registering a user inserts a row and returns the generated key:

```
INSERT INTO users (name, email, password_hash)
VALUES ('Test User', 'test@gmail.com', '$2b$10$...bcrypt hash...')
RETURNING user_id, name, email;
```

Output 7.2

| user_id | name | email |
|---|---|---|
| 1 | Test User | test@gmail.com |

**Read.** Crops of the Cereals category, using a subquery:

```
SELECT crop_name, season, water_requirement
FROM crop
WHERE crop_category_id = (SELECT crop_category_id FROM crop_category
                          WHERE slug = 'cereals')
ORDER BY crop_id;
```

Output 7.3

| crop_name | season | water_requirement |
|---|---|---|
| Rice (Paddy) | Kharif (June-Nov) is the main season; Rabi/summer rice possible with irrigation. | High |
| Wheat | Rabi season, sown October-December. | Moderate |
| Maize (Corn) | Grown in Kharif, Rabi and spring seasons depending on region. | Moderate |

**Update.** Changing a rainfall value, shown inside a transaction that is rolled back so that the reference data is not altered:

```
BEGIN;
UPDATE location SET annual_rainfall_mm = 2950 WHERE locality = 'Kole Lands';
SELECT locality, annual_rainfall_mm FROM location WHERE locality = 'Kole Lands';
ROLLBACK;
```

Output 7.4: UPDATE 1, then

| locality | annual_rainfall_mm |
|---|---|
| Kole Lands | 2950 |

**Delete.** Deleting a user removes that user's chat history through ON DELETE CASCADE (also run inside a transaction that is rolled back):

```
BEGIN;
DELETE FROM users WHERE email = 'test@gmail.com';
SELECT COUNT(*) FROM chat_history WHERE user_id = 1;
ROLLBACK;
```

Output 7.5: DELETE 1, then

| count |
|---|
| 0 |

## 7.6 Join, nested and aggregate queries

### Query A: crops suitable for Kole Lands (three-table join, ordered by suitability).

```
SELECT c.crop_name, s.suitability, s.reason
FROM location_crop_suitability s
JOIN crop c     ON c.crop_id = s.crop_id
JOIN location l ON l.location_id = s.location_id
WHERE l.locality = 'Kole Lands'
ORDER BY CASE s.suitability WHEN 'Highly Suitable' THEN 1
                           WHEN 'Suitable' THEN 2 ELSE 3 END, c.crop_name;
```

Output 7.6

| crop_name | suitability | reason |
|---|---|---|
| Rice (Paddy) | Highly Suitable | Waterlogged lowland and high rainfall favour rice |
| Banana | Suitable | Warm climate and fertile soils support banana |
| Coconut | Suitable | Humid climate supports coconut |
| Black Pepper | Moderately Suitable | Suitable climate but site drainage matters |

### Query B: livestock suitable for Kole Lands.

```
SELECT a.animal_name, s.suitability, s.reason
FROM location_livestock_suitability s
JOIN livestock a ON a.livestock_id = s.livestock_id
JOIN location l  ON l.location_id = s.location_id
WHERE l.locality = 'Kole Lands'
ORDER BY CASE s.suitability WHEN 'Highly Suitable' THEN 1 ELSE 2 END, a.animal_name;
```

Output 7.7

| animal_name | suitability | reason |
|---|---|---|
| Jersey Cow | Highly Suitable | Humid region with established dairy potential |
| Malabari Goat | Highly Suitable | Malabari goat is well adapted to Kerala |
| Duck (Khaki Campbell) | Suitable | Waterlogged lowlands support duck rearing |

### Query C: Kerala locations with rainfall above 2500 mm (selection with ordering).

```
SELECT locality, district, annual_rainfall_mm
FROM location
WHERE state = 'Kerala' AND annual_rainfall_mm > 2500
ORDER BY annual_rainfall_mm DESC;
```

Output 7.8

| locality | district | annual_rainfall_mm |
|---|---|---|
| Pokkali | Ernakulam | 3049 |
| High Ranges | Idukki | 3000 |
| Kole Lands | Thrissur | 2902 |

| locality | district | annual_rainfall_mm |
|---|---|---|
| Chavakkad | Thrissur | 2800 |
| Kuttanad | Alappuzha | 2746 |
| Central Midlands | Thrissur | 2600 |

### Query D: crop ratings per state (aggregation with GROUP BY).

```
SELECT l.state, COUNT(*) AS crop ratings
FROM location_crop_suitability s
JOIN location l ON l.location_id = s.location_id
GROUP BY l.state
ORDER BY crop_ratings DESC, l.state;
```

Output 7.9

| state | crop_ratings |
|---|---|
| Kerala | 14 |
| Maharashtra | 2 |
| Punjab | 2 |
| Rajasthan | 2 |
| West Bengal | 2 |

### Query E: locations that have no livestock rating (nested query with NOT EXISTS).

```
SELECT locality
FROM location l
WHERE NOT EXISTS (SELECT 1 FROM location_livestock_suitability s
                  WHERE s.location_id = l.location_id)
ORDER BY location_id;
```

Output 7.10

| locality |
|---|
| Central Midlands |
| Chavakkad |
| Chittur |
| Pokkali |
| Pollachi |
| Mysuru |
| Sri Ganganagar |

## 7.7 Back-end implementation

The back end is a single Express application (server.js) that connects to PostgreSQL through a connection pool. The pool reads either a DATABASE_URL (production) or separate host, user and password variables (local development), and enables TLS when a URL is used.

Table 7.2: REST API endpoints

| Method and endpoint | Purpose | Tables |
|---|---|---|
| POST /api/auth/signup | Register a user (password hashed) | users |
| POST /api/auth/login | Verify credentials and return a JWT | users |
| GET /api/ui/crop-categories | List crop categories | crop_category |
| GET /api/ui/crop-categories/:slug/crops | Crops of a category | crop, crop_category |

| Method and endpoint | Purpose | Tables |
|---|---|---|
| GET /api/ui/crops/:slug | Full crop guide | crop, crop_category |
| GET /api/ui/animal-categories (and /:slug/animals, /animals/:slug) | Livestock categories, animals and animal guide | livestock, livestock_category |
| GET /api/locations/states, /districts?state=, /localities?state=&district= | Fill the linked dropdowns | location |
| GET /api/recommendations/:locationId | Profile, ranked crops and ranked livestock of a location | location, both suitability tables, crop, livestock |
| POST /api/chat (token required) | Answer a question and store it | crop, livestock, location, chat_history |
| GET /api/chat/history (token required) | Return the user's past chats | chat_history |

**Registration and login** use bcrypt and JSON Web Tokens, and every query is parameterised:

```
const hash = await bcrypt.hash(password, 10);
const result = await pool.query(
  "INSERT INTO users (name, email, password_hash) VALUES ($1, $2, $3) " +
  "RETURNING user_id, name, email", [name, email, hash]);

const match = await bcrypt.compare(password, user.password_hash);
const token = jwt.sign({ user_id: user.user_id, email: user.email },
                       process.env.JWT_SECRET, { expiresIn: "7d" });
```

**Location recommendations** use the join queries of Section 7.6. The chat route first tries to find a crop name in the question, then an animal name, then a place name. It matches whole words, so that "rice" finds "Rice (Paddy)". It builds the reply from the matching row and stores the exchange:

```
await pool.query(
  "INSERT INTO chat_history (user_id, user_message, bot_response) VALUES ($1,$2,$3)",
  [req.user.user_id, message, reply]);
```

## 7.8 Front-end integration

All pages reach the server through one data-access file, data.js. The constant AG_API_BASE points to the deployed API. The helper apiGet() performs the request, and functions such as getCropCategories() and getRecommendationsForLocation() return the data in the shape the pages expect. The login result (name, email and token) is kept in the browser's localStorage, and agAuthHeaders() adds the token as an Authorization header for protected calls such as the chat.

## 7.9 Deployment

Table 7.3: Deployment configuration

| Component | Platform | Configuration |
|---|---|---|
| Database | Neon (PostgreSQL) | Project created on the free plan. Schema and data loaded with the SQL Editor and seed.js. Connection string uses sslmode=require. |
| API | Render web service (free) | Source: the agri-backend GitHub repository. Build command: npm install. Start command: npm start. Environment variables: DATABASE_URL and JWT_SECRET. The .env file is excluded from the repository by .gitignore. |
| Front end | GitHub Pages | Static files served from the Agriculture_Supporter repository. data.js points to the Render URL. The API allows this origin through CORS. |

# 8. Testing and Validation

Testing covered the application flow, the REST API and the database constraints. Cells shaded yellow are test cases that are defined here but whose actual output must be recorded by running the test.

## 8.1 Functional test cases

Table 8.1: Functional test cases

| ID | Test | Input | Expected output | Actual output | Result |
|---|---|---|---|---|---|
| T1 | Register a new user | Name, new email, password of 6 or more characters | Account created; row added to users with a bcrypt hash | Account created; row visible in pgAdmin with hash starting $2b$10$ | Pass |
| T2 | Login with valid credentials | Registered email and password | Login successful, token returned, dashboard opens | Token returned and dashboard opened | Pass |
| T3 | Login with wrong password | Registered email, wrong password | 401 "Invalid email or password" | Run and record | To run |
| T4 | Register with an existing email | Email already in users | 400 "Email already registered" | Run and record | To run |
| T5 | Register with a missing field | Name left empty | 400 "Name, email and password are required" | Run and record | To run |
| T6 | Password and confirm password differ | Two different passwords | Message "Passwords do not match."; no request sent | Run and record | To run |
| T7 | Open a module page without logging in | Open dashboard.html with browser storage cleared | Redirect to the login page | Run and record | To run |
| T8 | List crop categories | GET /api/ui/crop-categories | 9 categories | 9 categories returned (Cereals to Cash Crops) | Pass |
| T9 | Browse crop guide | Crop Production, Cereals, Rice | Guide of Rice with all sections | Guide displayed from the database | Pass |
| T10 | Browse animal guide | Animal Husbandry, Dairy, Jersey Cow | Guide of Jersey Cow with all sections | Guide displayed from the database | Pass |
| T11 | Location search with ratings | Kerala, Thrissur, Kole Lands | Profile (2902 mm, High water); Rice Highly Suitable; Banana, Coconut Suitable; Black Pepper Moderately Suitable; Jersey Cow and Malabari Goat Highly Suitable; Duck Suitable | Same result returned and displayed | Pass |
| T12 | Different location gives different result | Kerala, Idukki, High Ranges | Tea and Coffee Arabica Highly Suitable; Malabari Goat Suitable | Run and record | To run |
| T13 | Location without ratings | Tamil Nadu, Coimbatore, Pollachi | Message that no recommendations exist yet | Run and record | To run |
| T14 | AgriBot crop question | "tell me about rice" | Reply with climate, soil, fertiliser, irrigation, pests and harvest of rice | Reply with the rice details returned | Pass |
| T15 | AgriBot without a token | POST /api/chat with no Authorization header | 401 "Login required" | Run and record | To run |
| T16 | AgriBot unknown question | "capital of France" | Helpful default message | Run and record | To run |

| ID | Test | Input | Expected output | Actual output | Result |
|---|---|---|---|---|---|
| T17 | Cross-origin request from the local test server | Signup from http://127.0.0.1:5501 | Request allowed after the CORS rule is fixed | Blocked before the fix; signup succeeded after the fix | Pass |

## 8.2 Database constraint and data integrity tests

The following statements test the constraints. Each is expected to fail with the stated error, and the data must remain unchanged.

```
-- D1 duplicate email
INSERT INTO users (name, email, password_hash) VALUES ('X','test@gmail.com','h');
-- D2 NOT NULL
INSERT INTO users (email, password_hash) VALUES ('new@x.com','h');
-- D3 invalid foreign key
INSERT INTO crop (crop_category_id, crop_name) VALUES (999, 'Test');
-- D4 CHECK constraint
INSERT INTO recommendation (location_id, recommendation_type, title)
VALUES (1, 'plant', 'Test');
-- D5 duplicate pair
INSERT INTO location_crop_suitability (location_id, crop_id, suitability)
VALUES (1, 1, 'Suitable');
-- D6 delete a parent that is still referenced
DELETE FROM crop WHERE slug = 'rice';
```

Table 8.2: Constraint tests

| ID | Constraint tested | Expected PostgreSQL error | Actual output | Result |
|---|---|---|---|---|
| D1 | UNIQUE on users.email | duplicate key value violates unique constraint "users_email_key" | Run and record | To run |
| D2 | NOT NULL on users.name | null value in column "name" of relation "users" violates not-null constraint | Run and record | To run |
| D3 | Foreign key crop to crop_category | insert or update on table "crop" violates foreign key constraint | Run and record | To run |
| D4 | CHECK on recommendation_type | new row for relation "recommendation" violates check constraint | Run and record | To run |
| D5 | UNIQUE (location_id, crop_id) | duplicate key value violates unique constraint | Run and record | To run |
| D6 | Foreign key protects a referenced crop | update or delete on table "crop" violates foreign key constraint on table "location_crop_suitability" | Run and record | To run |
| D7 | ON DELETE CASCADE on chat_history | Deleting a user also deletes that user's chats (see Output 7.5) | Count of chats for the deleted user is 0 | Pass |
| D8 | Passwords are not stored in plain text | password_hash holds a bcrypt hash | Hash beginning $2b$10$ seen in the users table | Pass |

## 9. Results and Screenshots

This chapter presents the results of the implemented system. Each figure shows a screen or database output and the explanation below it says what it demonstrates.

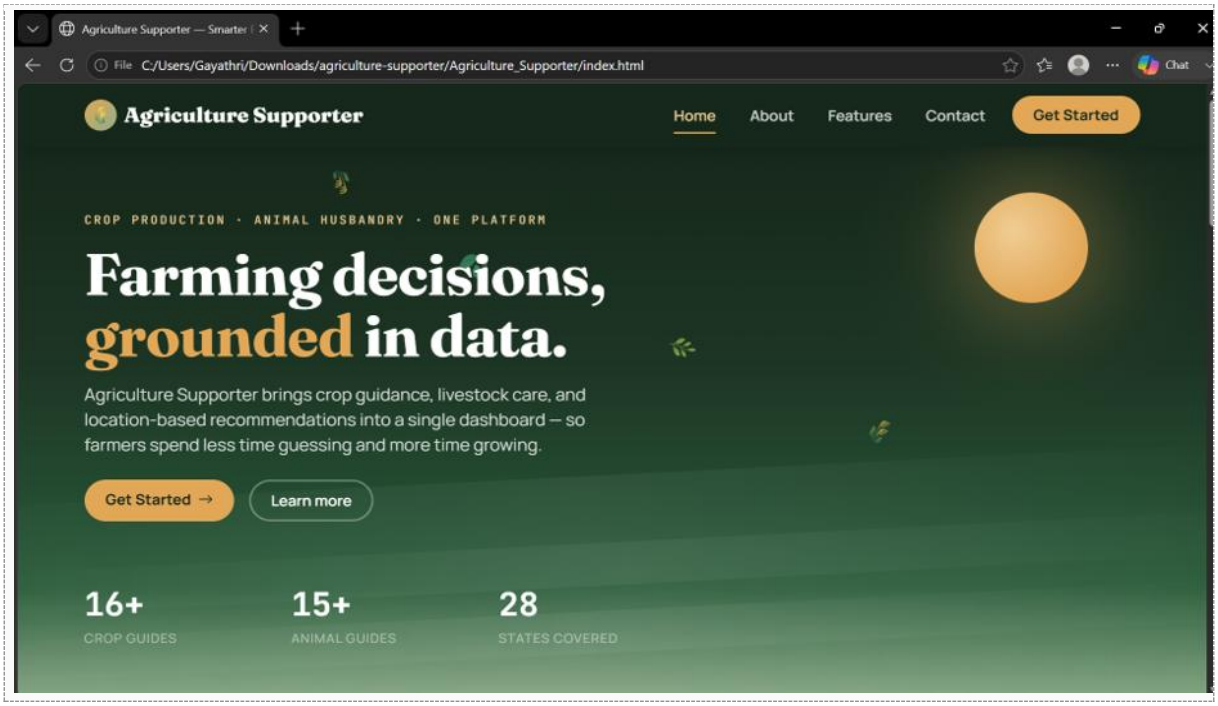



Figure 9.1: Landing page

The entry page introduces the project and leads the visitor to the login page through the Get Started button.

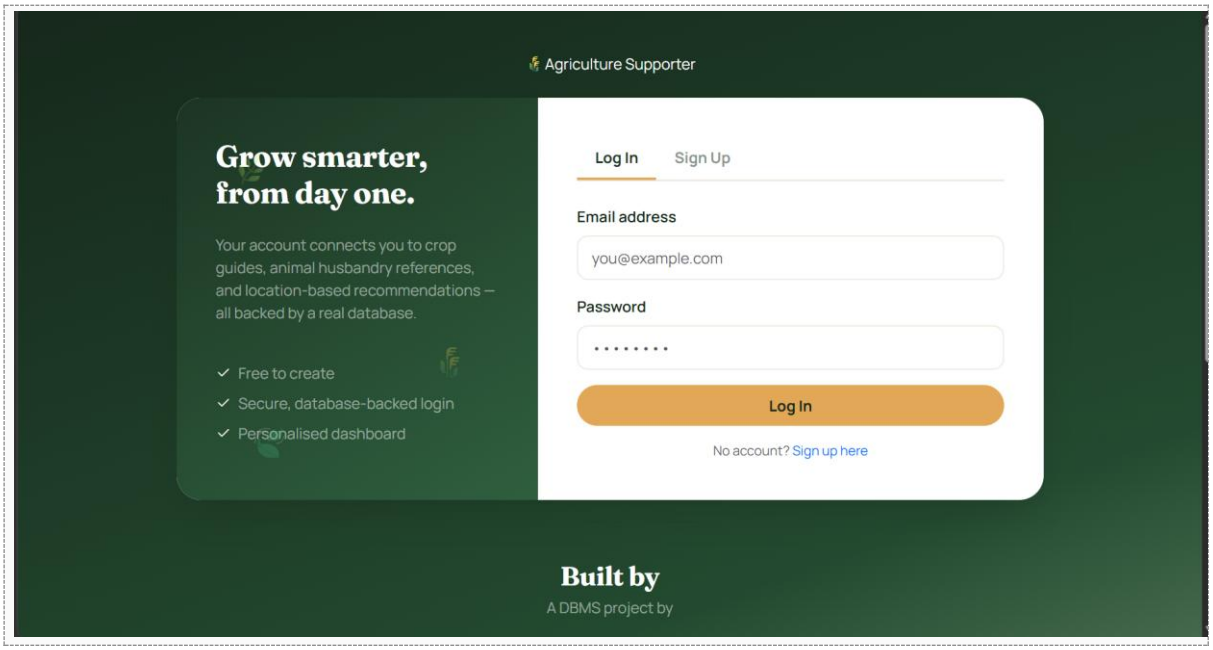



Figure 9.2: Sign Up form

A new user registers with name, email and password. Validation messages appear above the Create Account button.

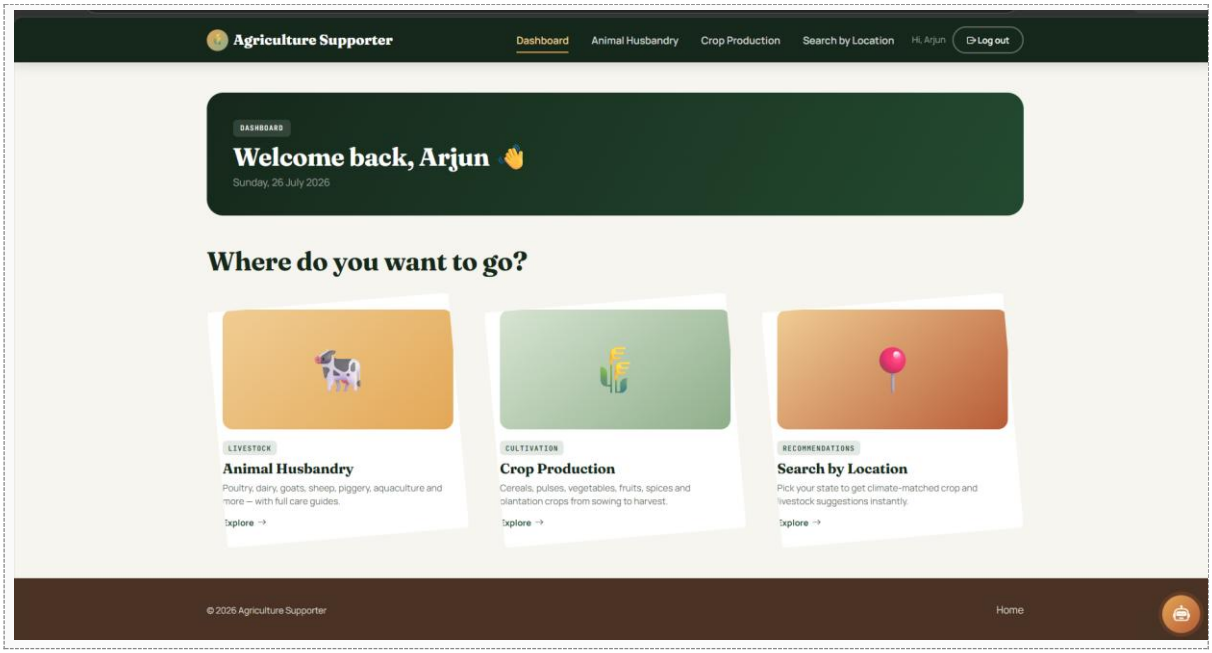



Figure 9.3: Dashboard after login

After a successful login the dashboard greets the user by name. The name comes from the user record returned by the database.

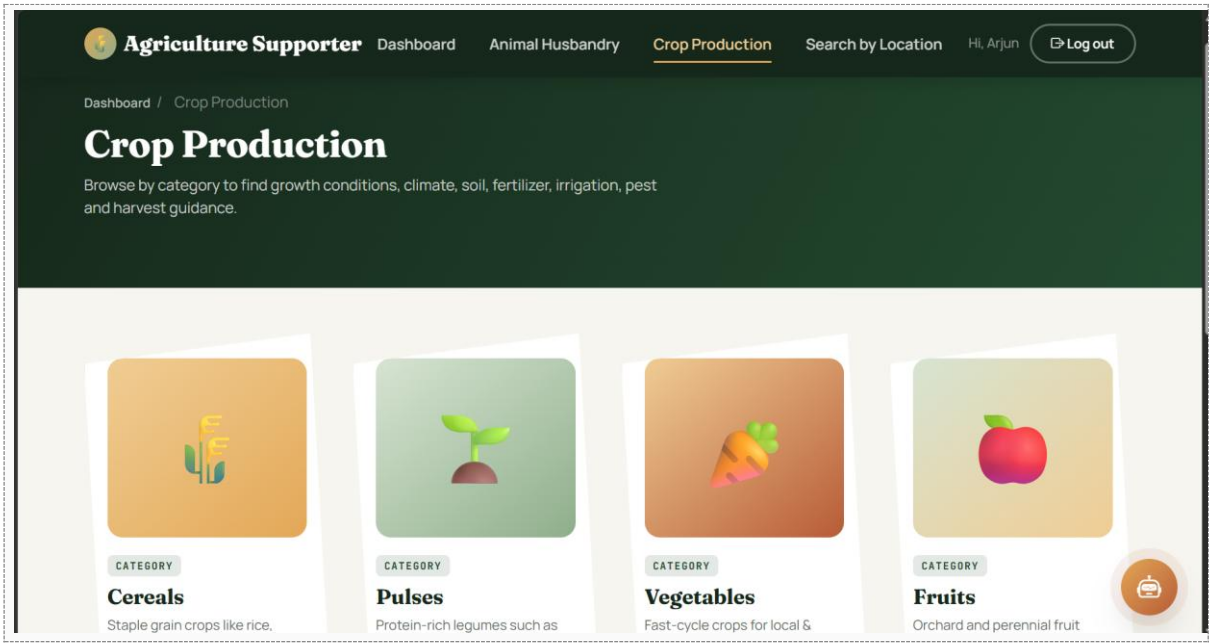



Figure 9.4: Crop Production categories

The nine category cards are loaded from the crop_category table. The Cash Crops card exists only in the database.

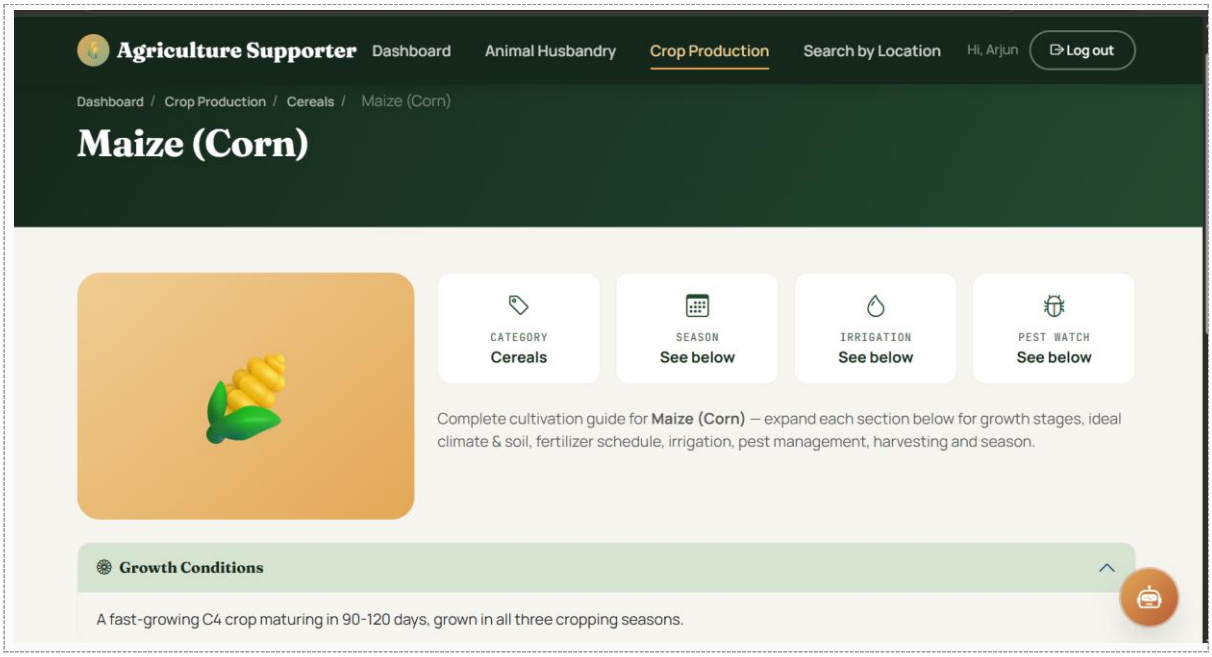



Figure 9.5: Crop guide for Maize

The full guide (growth, climate, soil, fertiliser, irrigation, pests, harvesting and season) is read from the crop table.

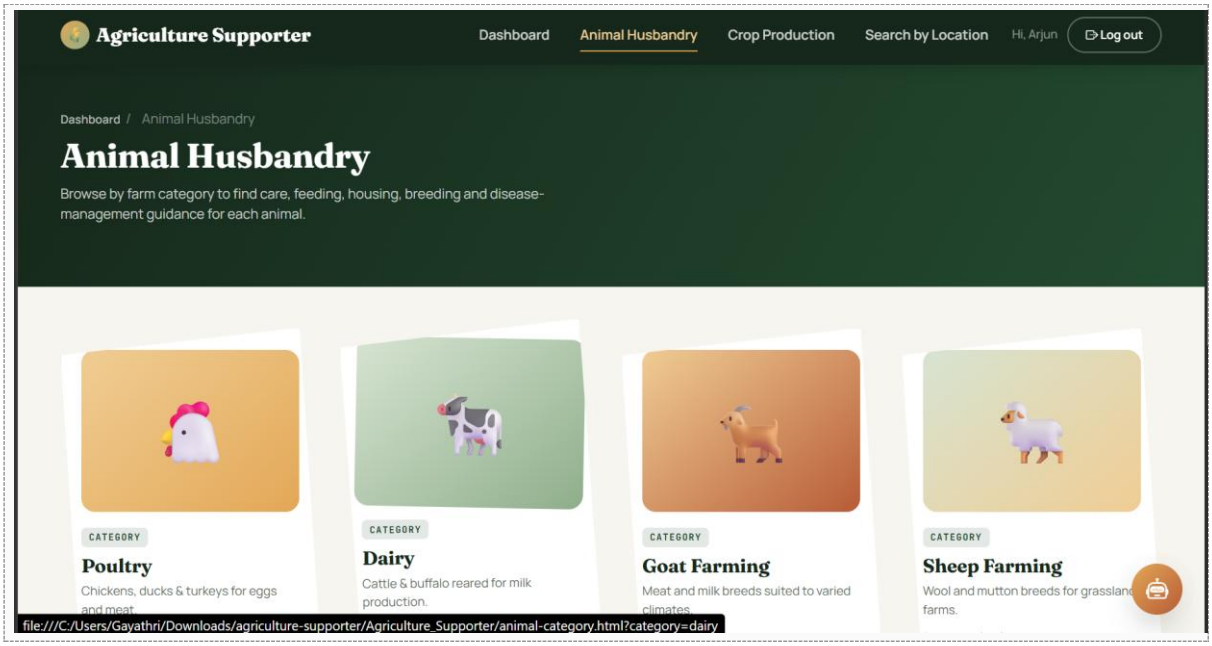



Figure 9.6: Animal Husbandry categories

The eight livestock categories are loaded from the livestock_category table.

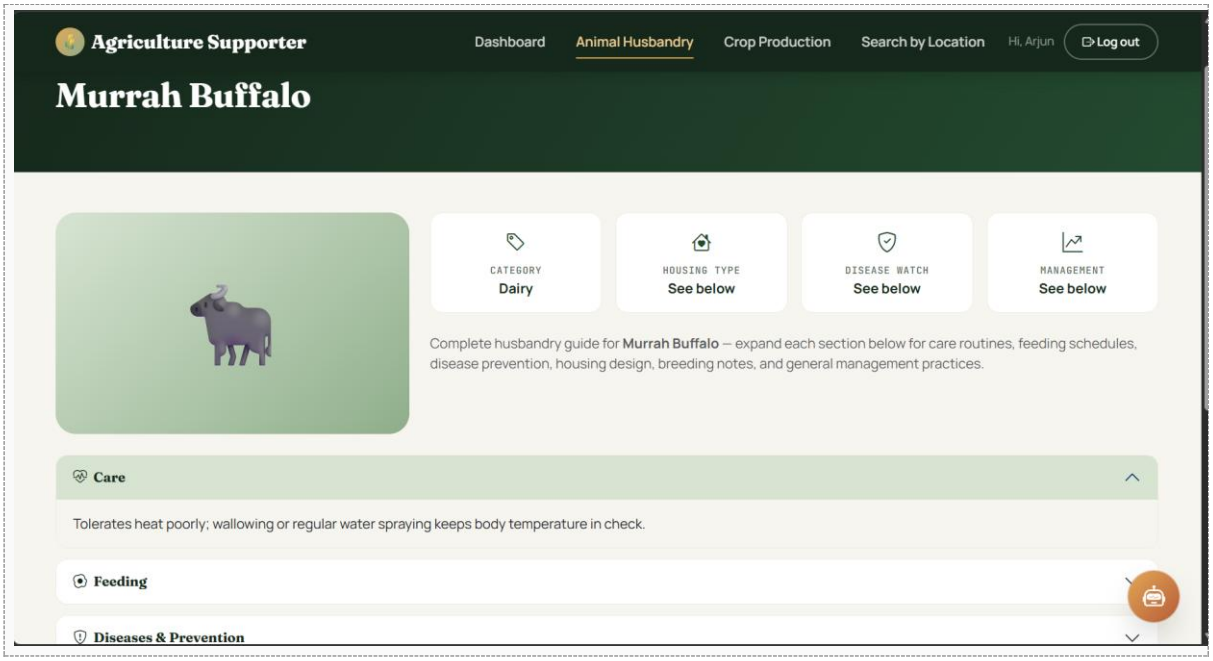



Figure 9.7: Animal guide for Murrah Buffalo

Care, feeding, diseases, housing, breeding and management details are read from the livestock table.

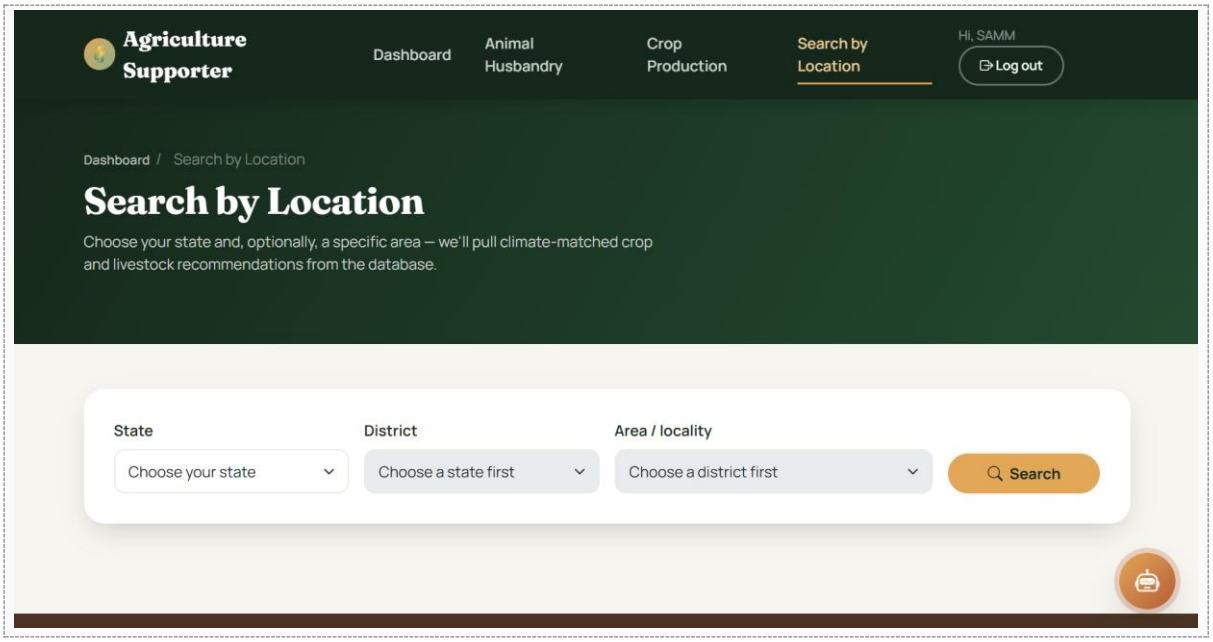



Figure 9.8: Linked location dropdowns

The district list depends on the chosen state and the area list depends on the chosen district, so only locations that exist in the database can be chosen.

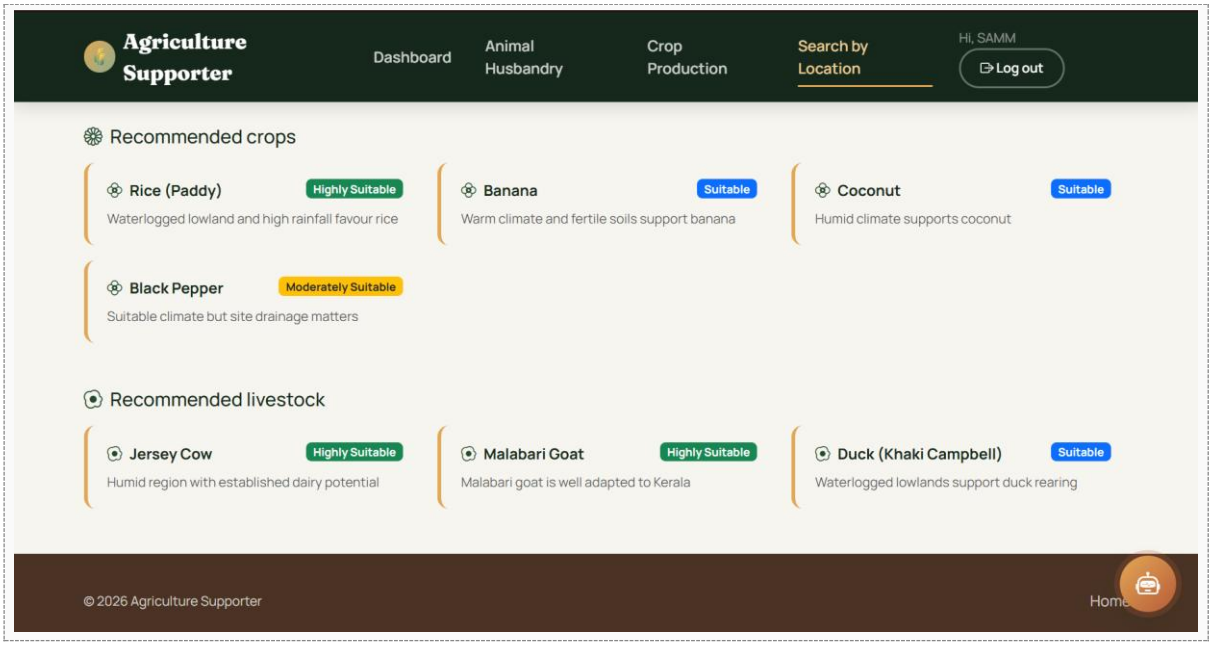



Figure 9.9: Recommendations for Kole Lands

The panel shows the agro-climatic profile and the ranked crops and livestock with suitability badges and reasons. Rice is rated Highly Suitable.

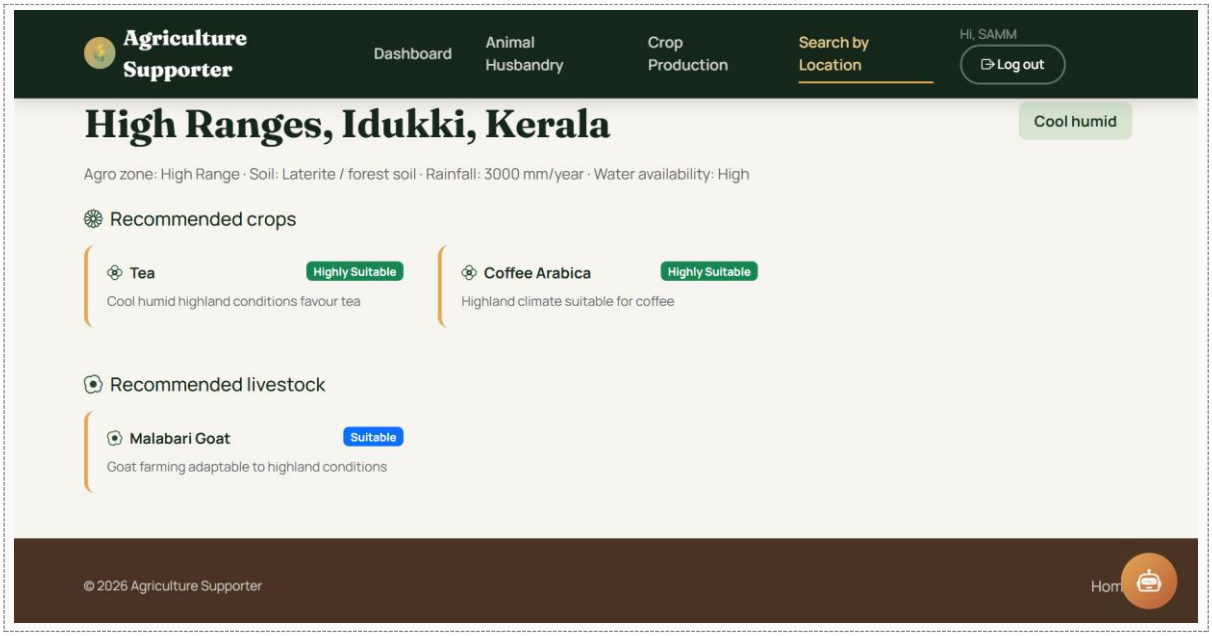



Figure 9.10: Recommendations for High Ranges

A different location gives a different result (Tea and Coffee Arabica), which shows that the recommendations depend on the location.

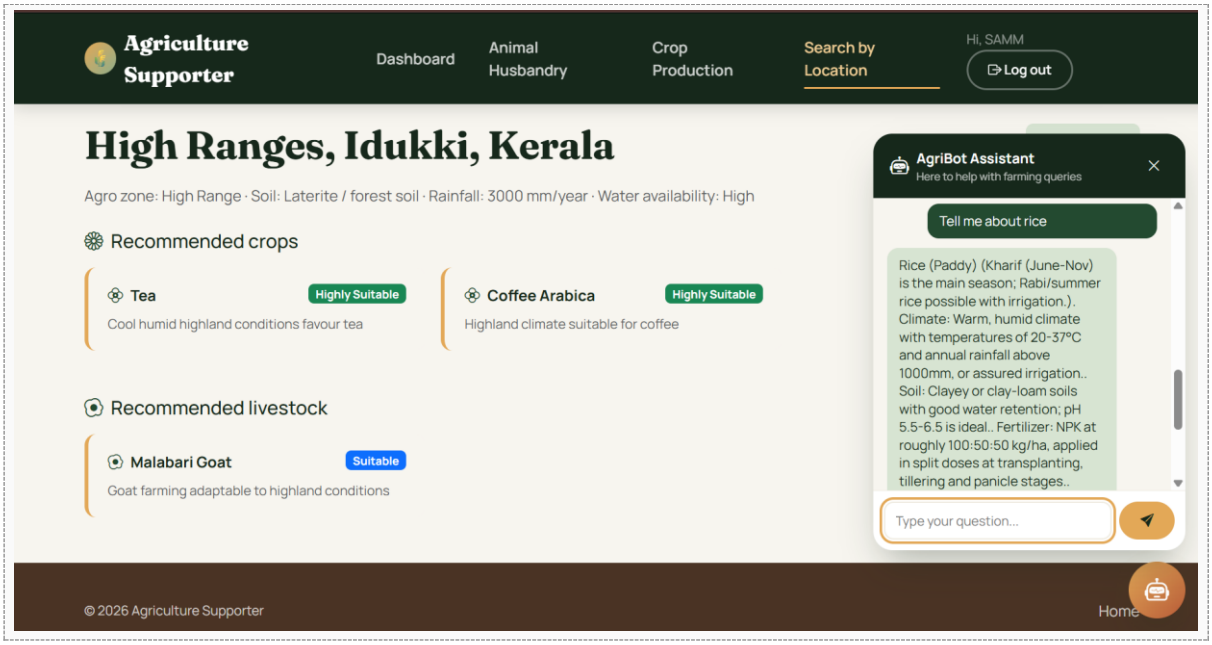



Figure 9.11: AgriBot conversation

The assistant answers from the database. Each question and answer is saved in chat_history.

*[ Paste screenshot here ]*

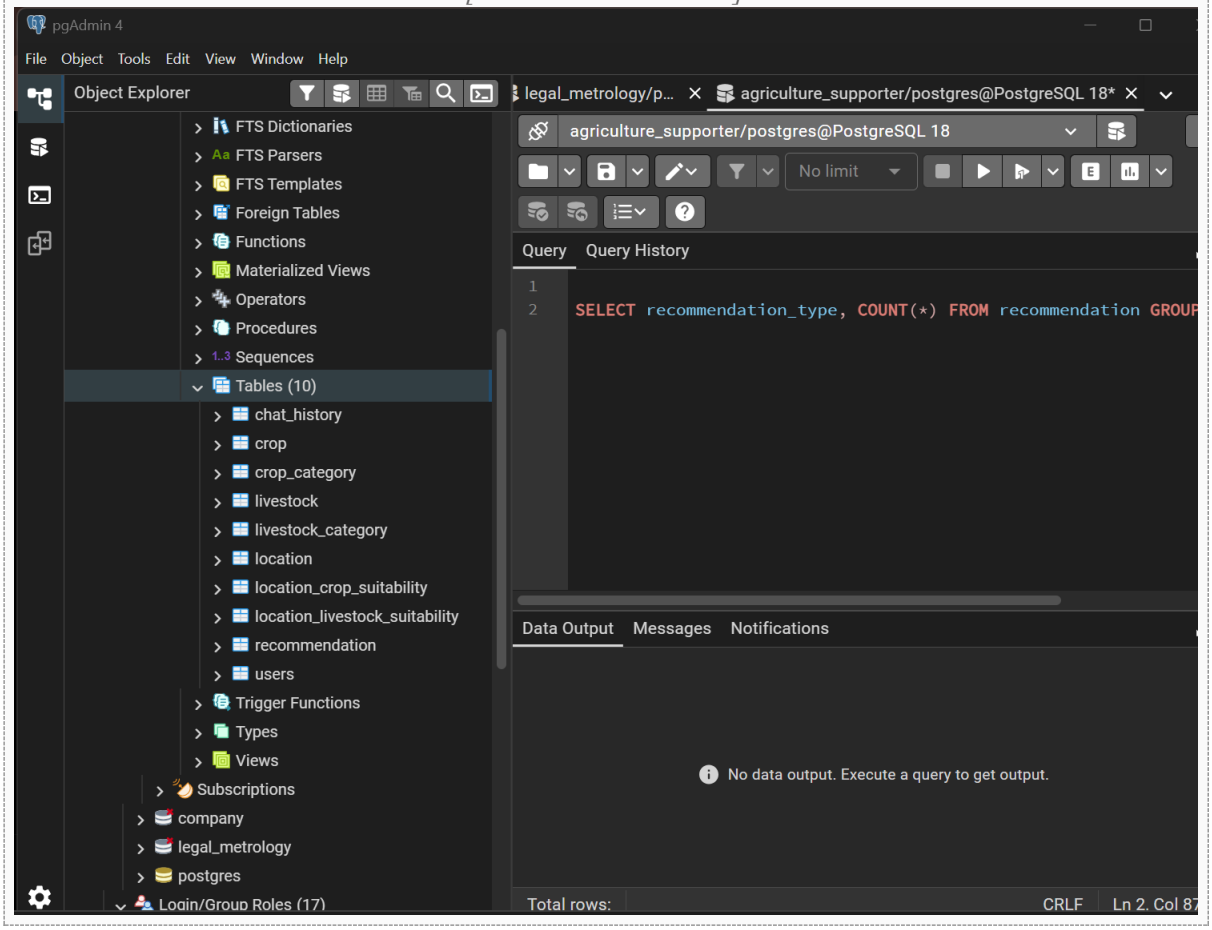



Figure 9.12: Tables in pgAdmin

The ten tables of the relational schema exist in the agriculture_supporter database.

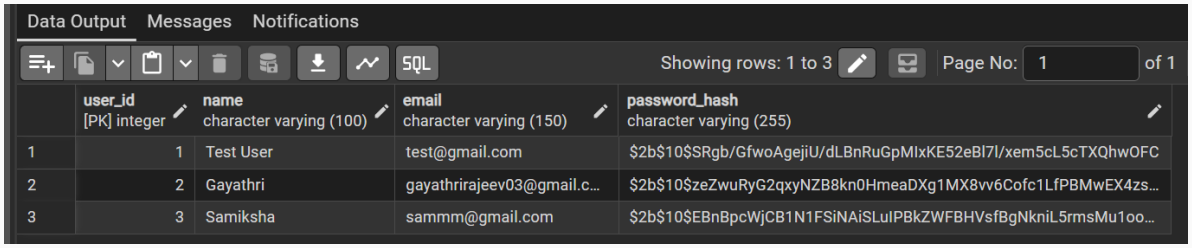



Figure 9.13: users table with hashed passwords

Passwords are stored as bcrypt hashes, never as plain text.

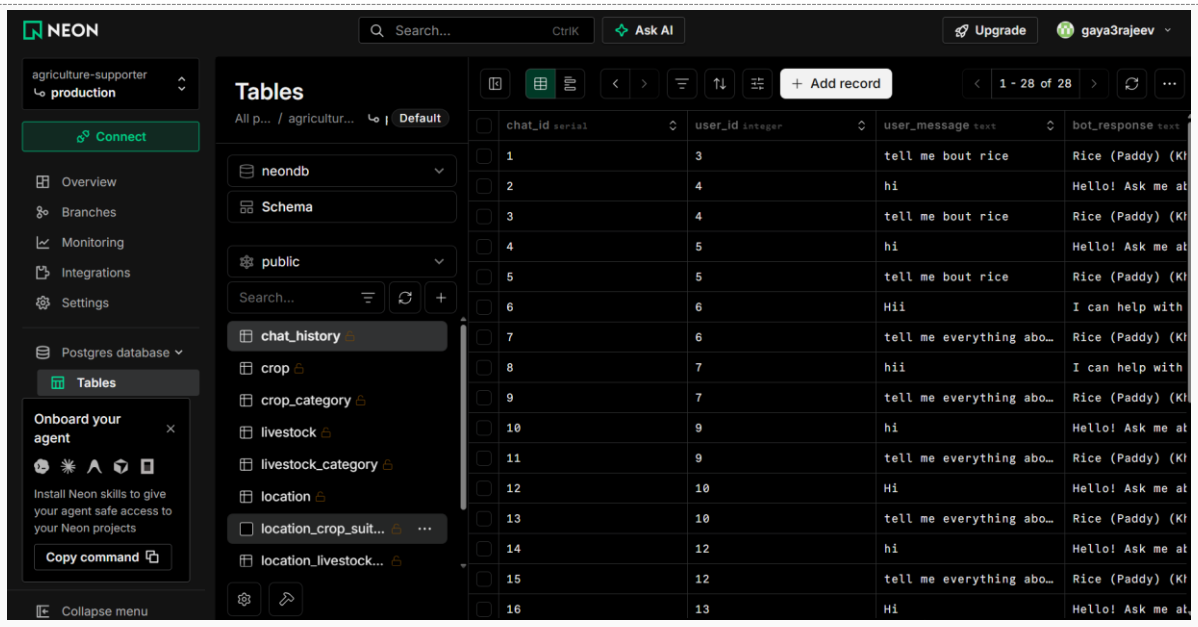



Figure 9.15: Neon SQL Editor and row counts

The deployed cloud database holds the same data (14 locations, 23 crops, 16 animals, 33 recommendations).

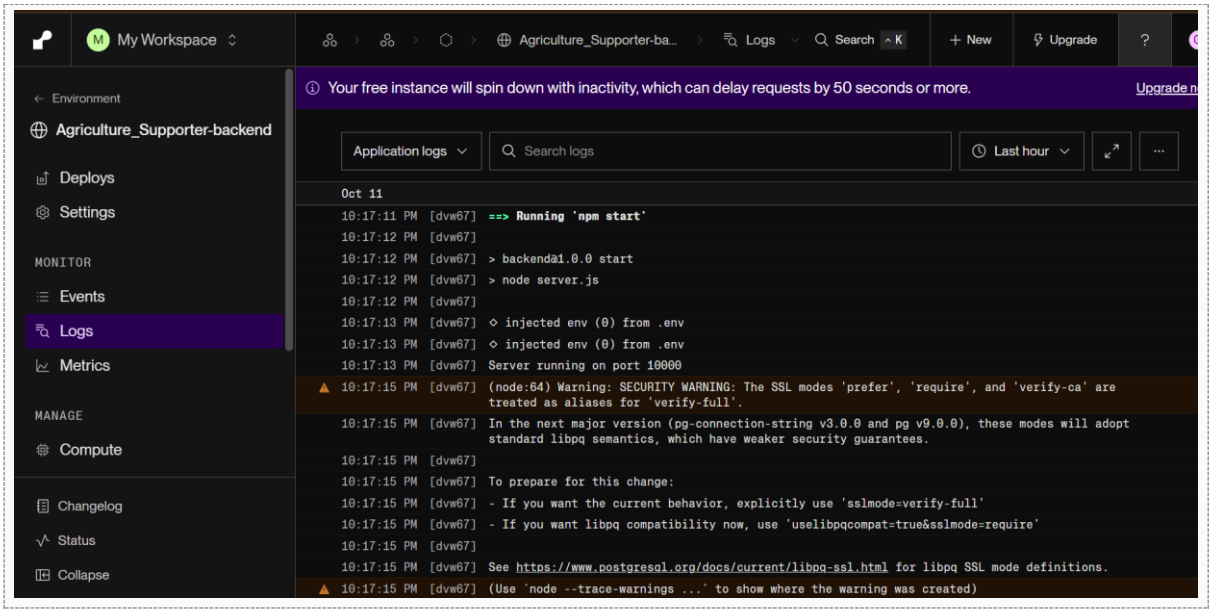



Figure 9.16: Render service running

The API is deployed and live. The live site is served from GitHub Pages and talks to this API.

# 10. Challenges Faced and Solutions

Table 10.1: Challenges and solutions

| Challenge | What happened | Solution |
|---|---|---|
| Incomplete ER model | The first ER diagram showed relationships such as TARGETS and USES but the RECOMMENDATION table had no foreign key to carry them. | Added nullable crop_id, livestock_id and user_id foreign keys and redrew the diagram. |
| Recommendations were not really location-specific | Storing crop descriptions alone could not answer "what suits this place?". | Added agro-climatic columns to LOCATION and two bridge tables that rate each crop and animal for each location with a reason. |
| Reserved word | A table named user conflicts with a PostgreSQL keyword. | Named the table users. |
| Column length errors | Loading the real data failed with "value too long for type character varying(40)" because descriptive text was longer than the column. | Changed those columns to TEXT. The seed transaction rolled back cleanly each time. |
| Front end data did not match the schema | The pages needed slugs, icons and colours and used hard-coded data. | Added slug, icon, color and blurb columns, loaded the same data with a seed script and replaced the hard-coded functions in data.js with API calls. |
| Chatbot did not recognise names | Names such as "Rice (Paddy)" did not match a question containing only "rice". | Matching now tries the full name, the name without brackets and the bracketed text, using whole-word matching. |
| Cloud database needs TLS | Connecting to Neon required SSL. | The connection pool uses the DATABASE_URL with sslmode=require and SSL enabled. |
| Browser blocked requests (CORS) | The test server on 127.0.0.1:5501 was blocked because only port 5500 was allowed. | Replaced the fixed list by a rule that allows the GitHub Pages site and any local development port. |
| Slow first request on free hosting | The free web service sleeps after 15 minutes of inactivity and needs up to a minute to wake. | The endpoint /api/test is opened a few minutes before use. A paid plan would remove the delay. |
| Secrets in a public repository | The repository is public, so passwords must not be uploaded. | Added .gitignore for .env and kept all secrets in environment variables on Render. |

# 11. Conclusion and Future Enhancements

## 11.1 Conclusion

Agriculture Supporter achieves its objectives. A relational database of ten tables was designed from an ER model, converted with clear mapping rules and normalised to third normal form, with one documented, deliberate denormalisation. The application provides secure registration and login, browsable crop and livestock guides, location-based recommendations that rank crops and animals with a reason for each rating, and a chat assistant whose answers and history are stored in the database. The complete system is deployed online using a PostgreSQL database on Neon, a Node.js API on Render and a front end on GitHub Pages.

The project demonstrates the main topics of the course: ER modelling, relational schema design, constraints, normalisation, SQL (DDL, DML, joins, subqueries and aggregation), transactions and integration of a DBMS with a web application.

## 11.2 Future enhancements

- Add more locations down to district and block level for all states, and refresh agro-climatic values from authoritative or live weather and soil sources.
- Build an administrator module with screens to add, update and delete crops, animals, locations and ratings, and add role-based access for administrators and farmers.
- Strengthen the database with a CHECK constraint on the suitability values, UNIQUE constraints on slug columns, indexes on frequently searched columns, views, stored procedures and triggers (for example an audit log).
- Replace the keyword-matching assistant by a retrieval-based assistant that uses the database as its knowledge source, and add Malayalam, Hindi and other regional languages.
- Add crop calendars, reminders, market price information and analytics dashboards.
- Improve security with refresh tokens, rate limiting and email verification.
- Provide a mobile application or progressive web app with offline access to saved guides.
- Move to a paid hosting plan to remove start-up delays, and add automatic backups and monitoring.

## References

[1] PostgreSQL Global Development Group, "PostgreSQL Documentation," https://www.postgresql.org/docs/ (accessed Oct. 2026).

[2] OpenJS Foundation, "Node.js Documentation," https://nodejs.org/docs/ (accessed Oct. 2026).

[3] "Express - Node.js web application framework," https://expressjs.com/ (accessed Oct. 2026).

[4] Neon, "Neon Documentation: Serverless Postgres," https://neon.com/docs (accessed Oct. 2026).

[5] Render, "Render Documentation: Deploy for Free," https://render.com/docs/free (accessed Oct. 2026).

[6] "Bootstrap 5.3 Documentation," https://getbootstrap.com/docs/5.3/ (accessed Oct. 2026).

[7] Indian Council of Agricultural Research (ICAR), National Bureau of Soil Survey and Land Use Planning, publications on agro-ecological regions of India (background for the location profiles; the values stored in the project are demonstration values compiled by the team).

[8] Government of Kerala, Department of Agriculture Development and Farmers' Welfare, material on the agro-ecological units of Kerala, including the Kole lands (background for the Kerala locations).